CS 251 · INTEGRATIVE PROJECT 1 · CLASS 5 OF 15

Python & OOP (Object-Oriented Programming) Refresher + Design

Blueprint Before Bricks

Friday, October 23, 2026 · Phase 2 · Midterm phase

Lecture 1 h in four parts · workshop 2 h · Brick 5 due Thursday, October 29, 2026, 23:59

5.1 Classes & dataclasses | **5.2 Collections & file parsing** | **5.3 Responsibilities & class diagrams** | **5.4 Abstract classes & clean code**

Use of these slides is subject to the **Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy** (click to open).

OPENING

Phase 2 starts today: the plan of the hour



0:00 1:00 → workshop

`0:00–0:11`
5.1 Classes, objects & @dataclass

`0:15–0:26`
5.2 Collections & CSV (Comma-Separated Values) parsing

`0:30–0:41`
5.3 Responsibilities → classes → UML (Unified Modeling Language)

`0:45–0:56`
5.4 ABCs (Abstract Base Classes), polymorphism & clean code

`0:11–0:15`
Activity 1 — two properties

`0:26–0:30`
Activity 2 — visualization + fun quiz

`0:41–0:45`
Activity 3 — zen + six UML cards

`0:56–1:00`
Activity 4 — spot the four smells

1:00–3:00 — In-class workshop, Brick 5 “Design Pack + Working Loader”. Bricks 1–4 closed Phase 1 (Project Proposal & Requirement Analysis); from today every Brick adds working code. Due Thursday, October 29, 2026, 23:59.

OPENING

By the end of this class you can...

1. Draw the **UML** (Unified Modeling Language) class diagram of JARVIS and defend the responsibility of each class.

   Brick 5 deliverable

2. Load the whole dataset into a `list[TimeEntry]` without crashing on a bad row.

   Brick 5 deliverable

3. Explain why `Tool` and `Provider` are abstract base classes and where the polymorphism is.

   Mid-Term Part B

4. Recognize four Python code smells that cost marks.

   Mid-Term Part B

5.1 Classes, objects & @dataclass — key terms

Class

A named definition that groups data (**attributes**) and the operations on that data (**methods**). In Python a class is created by the `class` statement; `TimeEntry` is a class.

Object (instance)

One concrete value created from a class. It has its own attribute values and the methods of its class. `first = TimeEntry(...)` creates one object; `type(first)` is `TimeEntry`.

Method

A function defined inside a class. Python passes the object on which it is called as the first parameter, named `self` by convention.

`@dataclass`

The decorator of the standard-library module `dataclasses` (Python 3.7, **PEP** (Python Enhancement Proposal) 557) that writes `__init__`, `__repr__` and `__eq__` from the annotated fields of a class.

`__post_init__`

A method that the generated `__init__` calls after it has stored the fields; it is the place for validation.

Encapsulation

Keeping data together with the code that uses it in one class, and controlling how that data can be read or changed from outside the class.

Why one row of the log needs a class

- One **CSV** (Comma-Separated Values) row = nine **str** values: `'2026-09-14'`, `'S1'`, … `'36'`, `'manual'`, `'1'`
- Text cannot be added (minutes), compared (dates) or checked (−19 minutes)
- Bricks 3–4: lists of dicts — acceptable for **one** script
- JARVIS: two engines, a store, a reporter, several tools use the **same** rows
- → define the row **once**, as a class: `TimeEntry`



KEY IDEA

Nine pieces of text cannot be added, compared or checked; one TimeEntry class used by every module can.

Class and object

- **Class**: the definition — attributes + methods
- **Object** (instance): one value built from it
- `first` and `pilot`: two objects, same attribute names, own values
- `self` = the object being initialised

```
class TimeEntryPlain:
    def __init__(self, date, staff_id, routine, minutes, tool):
        self.date = date
        self.staff_id = staff_id
        self.routine = routine
        self.minutes = minutes
        self.tool = tool
```

The @dataclass decorator

- A **decorator** (`@...`) receives the class and returns a modified class
- `@dataclass`: Python 3.7, **PEP** (Python Enhancement Proposal) 557

- Each `name: type` line = one **field** + a **type hint**
- Type hints are **not** checked at run time

```python
from dataclasses import dataclass
from datetime import date


@dataclass(frozen=True)
class TimeEntry:
    """One timed run of one routine by one staff member (one row of the time log)."""

    date: date
    staff_id: str
    routine: str
    category: str
    start_hour: int
    minutes: int
    interruptions: int
    tool: str
    done: bool
```




What @dataclass(frozen=True) writes for TimeEntry

Table 5.1.A — What @dataclass(frozen=True) generates · file tables/t51_dataclass_methods.csv

Method written by @dataclass	What it does for TimeEntry
`__init__`	Takes the nine fields as parameters, in the order of the class body, and stores them in the new object.
`__repr__`	Returns `TimeEntry(date=datetime.date(2026, 9, 14), staff_id='S1', …, done=True)`: the class name and every field.
`__eq__`	Compares two entries field by field: two objects with the same nine values are equal.
`__setattr__`, `__delattr__` (frozen)	Raise `FrozenInstanceError` on any attempt to change or delete a field after creation.
`__hash__` (frozen and eq)	Computes a hash from the fields, so an entry can be stored in a set or used as a dictionary key.

Validation in __post_init__

- The generated `__init__` calls `__post_init__` after storing the fields
- Rules from the Brick 2 data dictionary
- `raise ValueError(...)` → no object is created
- **Invariant**: true for every `TimeEntry`, for its whole life

```
    def __post_init__(self) -> None:
        if self.minutes < 0:
            raise ValueError(f"minutes must be >= 0, got {self.minutes}")
        if not 0 <= self.start_hour <= 23:
            raise ValueError(f"start_hour must be 0-23, got {self.start_hour}")
        if self.tool not in ALLOWED_TOOLS:          # {"manual", "template", "jarvis"}
            raise ValueError(f"unknown tool {self.tool!r}")
```




Methods and properties

- `first.cost(28.0)` → 36 / 60 × 28 = **16.80 $**
- The hourly value is a **parameter**, never a number in the method
- **Property** (`@property`): read like an attribute, computed when read
- `first.hours` → **0.6**

```
    def cost(self, hourly_value: float) -> float:
        return self.minutes / 60 * hourly_value

    @property
    def hours(self) -> float:
        return self.minutes / 60
```




Encapsulation in Python

- **Encapsulation**: data + the code that uses it, in one class; the class controls access
- No `private` keyword in Python
- `_tools` — one underscore: **internal**, by convention (**PEP** (Python Enhancement Proposal) 8)
- `__secret` — two underscores: **name mangling** → `_ClassName__secret`
- Property without setter: assignment → `AttributeError` (frozen class: `FrozenInstanceError`)
- `frozen=True`: every field read-only after creation
- **UML** (Unified Modeling Language) (Lesson 5.3): **–** private, **+** public



KEY IDEA

Python uses conventions: _name is internal, __name is renamed, properties and frozen=True control changes.



__repr__ and __str__

- `repr(obj)` → `__repr__`: unambiguous, for developers
- `print(obj)` → `__str__`: readable, for people
- No `__str__`? `print()` falls back on `__repr__`

```
>>> repr(first)
"TimeEntry(date=datetime.date(2026, 9, 14),
    staff_id='S1', routine='email_triage',
    category='communication', start_hour=9,
    minutes=36, interruptions=1,
    tool='manual', done=True)"
>>> print(first)
2026-09-14 S1 email_triage: 36 min (manual)
```




Activity 1 — Add two properties (0:11–0:15)

⏱️ 0:11–0:15 · Short practical

1. Add a property `hours` that returns `self.minutes / 60`.
2. Add a property `is_automated` that returns `True` when the tool is `"jarvis"`.
3. Test in the **REPL** (Read–Eval–Print Loop): `first.hours` must give `0.6`; `pilot.is_automated` must give `True`.
4. One sentence: why is `hours` a property and not a tenth field?



Videos for home — Lesson 5.1

For home, not for class. Each video is public and shorter than 15 minutes (checked October 17, 2026); videos are third-party content — sections 14 and 15 of the Cyber.SoHo Legal Notice apply.

Learn Python Object Oriented Programming!

Bro Code · YouTube · 12:18 · youtube.com/watch?v=1XE-_s4ZBT8

Classes, objects, __init__, self, attributes and methods, from the beginning — the base of Lesson 5.1.

If You're Not Using Python DATA CLASSES Yet, You Should

ArjanCodes · YouTube · 10:55 · youtube.com/watch?v=vRVVyI9uaZc

@dataclass, field(), ordering, frozen=True and __post_init__ — the exact tools of TimeEntry.

Python OOP (Object-Oriented Programming) Tutorial 6: Property Decorators - Getters, Setters, and Deleters

Corey Schafer · YouTube · 9:33 · youtube.com/watch?v=jCzT9XFZ5bw

@property, and how a property controls access to data (Activity 1 and encapsulation).

Python OOP Tutorial 5: Special (Magic/Dunder) Methods

Corey Schafer · YouTube · 13:50 · youtube.com/watch?v=3ohzBxoFHAY

__repr__ versus __str__ and other special methods, as used by print(first) and repr(first).

Real-life example 5.1 — one row, one checked object

THE TASK

Create `TimeEntry` objects for two real rows of `boreal_routines_2026.csv` — S1's manual e-mail triage of September 14 (36 minutes) and S1's first assistant-triaged run of September 25 (19 minutes) — compute their cost at 28 $/h, and show that two damaged rows (−19 minutes, tool `jarvs`) are refused and that an object cannot be changed after creation.

Demo: `code/time_entry.py` · interactive replay: `code/demo51_time_entry.html`

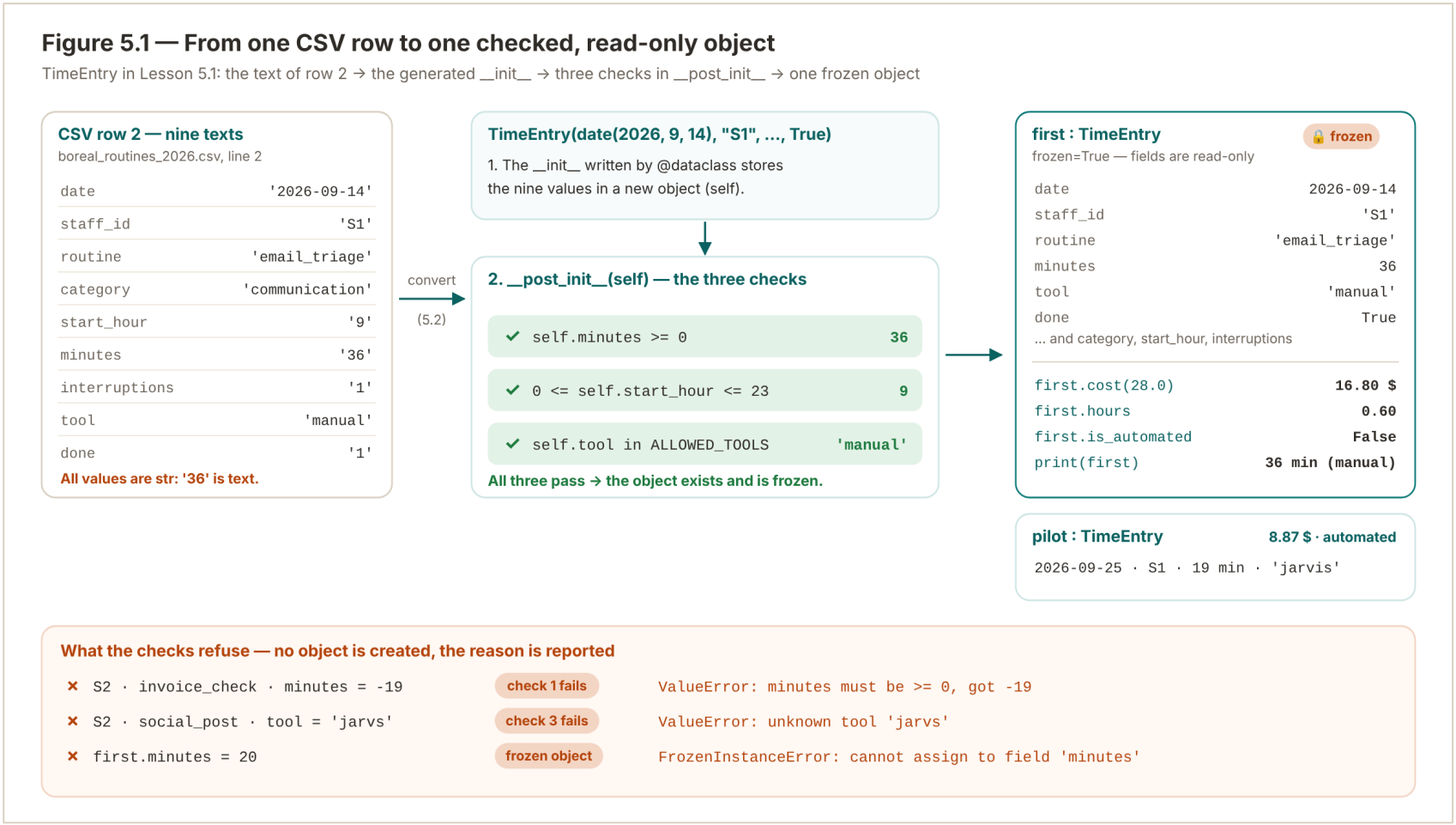
DELIVERABLES OF THE EXAMPLE

- `jarvis/models.py` with `TimeEntry`: nine fields, three checks, `cost()`, `hours`, `is_automated`, `__str__`
- The two costs checked by hand: 16.80 $ and 8.87 $
- The three rejection messages, written down: they become three `pytest` tests

WHERE IT RETURNS (FUTURE BRICKS)

- Brick 5 (due Oct 29) — `jarvis/models.py`, tag alpha-0.1
- Brick 6 (due Nov 5) — `EconEngine.routine_cost` uses `TimeEntry.cost`
- Brick 9 (due Dec 3) — tests with `pytest.raises(ValueError)`
- Brick 10 (due Dec 10) — input validation demonstrated
- Mid-Term Part B (Nov 20) — explain encapsulation on this code

Figure 5.1 — From one CSV (Comma-Separated Values) row to one checked object



WHAT THE FIGURE SHOWS

The text values of row 2 of the dataset become the nine fields of a TimeEntry object. The generated __init__ stores them and calls __post_init__; three checks decide whether the object exists. The two damaged rows of S2 stop at the check that fails. Methods and properties then work on the object; frozen=True refuses later changes.

Files: figures/fig51_timeentry_object.svg and figures/fig51_timeentry_object.png

5.2 Collections & CSV (Comma-Separated Values) parsing — key terms

Collection

An object that contains other objects. The four built-in collections used in JARVIS are `list`, `dict`, `set` and `tuple`.

`csv.DictReader`

A reader of the standard-library module `csv` that returns each data line as a dict mapping the column names of the header to text values.

`pathlib.Path`

An object that represents a file-system path. The operator `/` joins parts with the separator of the operating system.

Exception

An object that signals an error and interrupts the normal order of execution: `ValueError` (right type, wrong value), `TypeError` (wrong type, such as `None`), `KeyError` (missing key).

`try` / `except`

A statement that runs a block and, if one of the named exceptions occurs, runs the handler instead of stopping the program.

`defaultdict`

A dict subclass of the module `collections` that creates a default value, such as an empty list, the first time a missing key is used.



Four collections, four jobs

Table 5.2.A — The four built-in collections and their job in the loader · file tables/t52_collections.csv

Type	Ordered	Changeable	Duplicates	Job in JARVIS
`list`	yes	yes	allowed	`entries`: the 420 `TimeEntry` objects, in file order
`dict`	insertion order	yes	keys unique	one **CSV** (Comma-Separated Values) row: `{"date": "2026-09-14", "staff_id": "S1", …}`
`set`	no	yes	no	`ALLOWED_TOOLS`; the staff **IDs** (identifiers) `{"S1", …, "S5"}`
`tuple`	yes	no	allowed	one skipped line: `(4, "invalid literal for int() with base 10: '17.0'")`



Paths: pathlib, never a hard-coded path

- `C:\Users\ana\Desktop\boreal.csv` → one computer only
- Fails on a Mac, on GitHub Actions (Linux), at the defense
- Rubric, Lab Proficiency: “hard-coded paths” = **Developing**
- `pathlib`: Python 3.4, **PEP** (Python Enhancement Proposal) 428 — `/` joins parts

```
from pathlib import Path

HERE = Path(__file__).resolve().parent      # the folder that contains this .py file
DATA = HERE.parent / "data"                 # the data folder next to the code folder
csv_path = DATA / "boreal_routines_2026.csv"
```

Opening the file and csv.DictReader

- `with` → the file is closed, even after an error
- `newline=""` → required by the **csv** module
- `encoding="utf-8"` → the é of Boréal on every system
- `DictReader`: header = keys, **every value is str**
- Extra value → key `None` (restkey) · missing value → `None` (restval)
- Empty lines are skipped

```
with path.open(newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        ...

# line 2 of the dataset:
{"date": "2026-09-14", "staff_id": "S1", ...,
 "minutes": "36", "tool": "manual", "done": "1"}
```




Converting text — and what fails (1/2)

Table 5.2.B — int() (1/2) · file tables/t52_conversions.csv

Text in the file	Conversion	Result
`"36"`	`int()`	`36`
`" 36 "`	`int()`	`36` (surrounding spaces are accepted)
`"17.0"`	`int()`	`ValueError: invalid literal for int() with base 10: '17.0'`
`""` (empty cell)	`int()`	`ValueError: invalid literal for int() with base 10: ''`
`None` (missing field)	`int()`	`TypeError: int() argument must be a string, …, not 'NoneType'`



Converting text — and what fails (2/2)

Table 5.2.B — date and done (2/2) · file tables/t52_conversions.csv

Text in the file	Conversion	Result
`"2026-09-14"`	`date.fromisoformat()`	`date(2026, 9, 14)`
`"2026/09/14"`	`date.fromisoformat()`	`ValueError: Invalid isoformat string: '2026/09/14'`
`"1"`	`== "1"`	`True`

- **ISO** (International Organization for Standardization) 8601: `yyyy-mm-dd` → `date.fromisoformat`
- **EAFP** (Easier to Ask for Forgiveness than Permission): try the conversion, handle the exception
- `int(float("17.9"))` → 17 without warning: **rejected** instead



try/except: report the row, never crash

- Catch only **ValueError** and **TypeError** (bad row)
- `FileNotFoundError`, missing column → **stop** (setup problem)
- `enumerate(reader, start=2)` → the line number (no blank lines in our exports; else `reader.line_num`)
- Sample: loaded **8**, skipped **4** · full file: **420** / **0**

```
for line_no, row in enumerate(reader, start=2):      # line 1 of the file is the header
    try:
        entries.append(self._parse_row(row))          # conversion + TimeEntry checks
    except (ValueError, TypeError) as err:
        self.skipped.append((line_no, str(err)))      # keep the line number and the reason
```

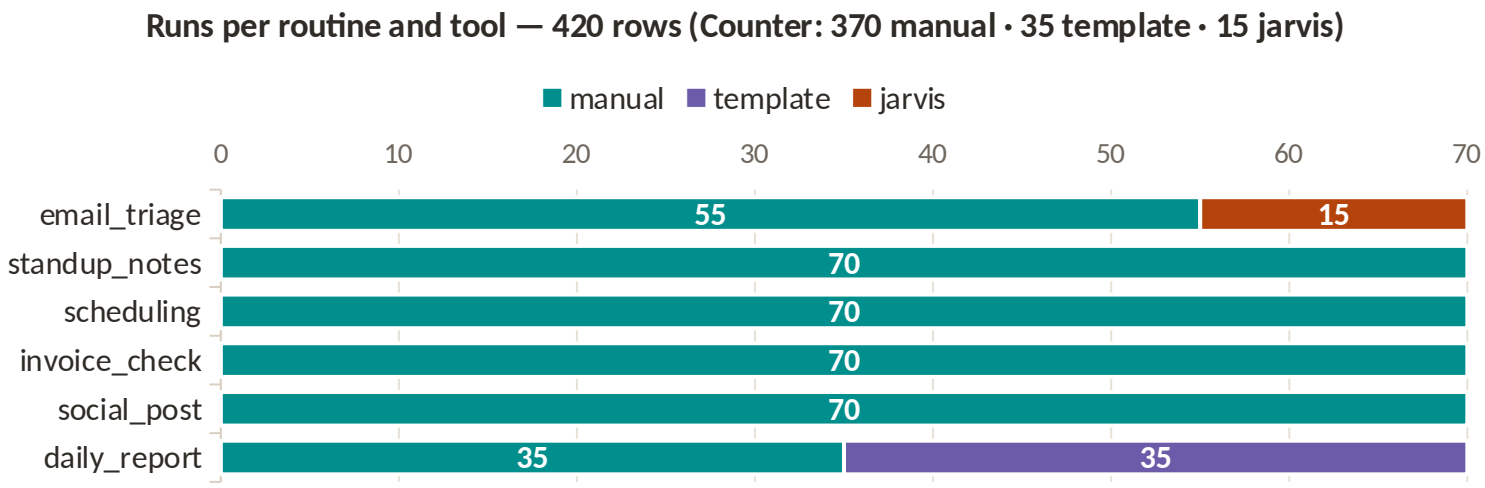



Grouping with defaultdict, counting with Counter

```python
from collections import Counter, defaultdict

minutes_by_routine = defaultdict(list)
for entry in entries:
    minutes_by_routine[entry.routine].append(entry.minutes)   # 6 keys x 70 values

Counter(entry.tool for entry in entries)    # {'manual': 370, 'template': 35, 'jarvis': 15}
```



KEY IDEA

defaultdict(list) groups the minutes per routine (6 keys × 70 values); Counter counts the tools in one line. Only e-mail triage (jarvis) and the daily report (template) are automated so far.

Activity 2 — The path of one row + fun quiz (0:26–0:30)

⏱️ 0:26–0:30 · Visualization + fun quiz

1. Show Figure 5.2 (file → row → convert → object → list) and follow line 4 of the sample through it.
2. Fun quiz 1 — What does `DictReader` give you for each data line?
3. Fun quiz 2 — What breaks on a trailing comma?
4. Fun quiz 3 — Why does `int("12.0")` fail?

Videos for home — Lesson 5.2

For home, not for class. Each video is public and shorter than 15 minutes (checked October 17, 2026); videos are third-party content — sections 14 and 15 of the Cyber.SoHo Legal Notice apply.



CSV (Comma-Separated Values) Files with Python — Reading and Writing

teclado · YouTube · 10:09 · youtube.com/watch?v=W7QByFjVom8

The csv module, DictReader and DictWriter, and why splitting lines on commas is not enough.



Python Tutorial: Using Try/Except Blocks for Error Handling

Corey Schafer · YouTube · 10:34 · youtube.com/watch?v=NIWwJbo-9_8

Catching specific exceptions with try/except/else/finally — the pattern of DataLoader.load().



The Right Way to Handle File Paths in Python: pathlib.Path Explained For Beginners

Koolac · YouTube · 9:57 · youtube.com/watch?v=P_aO97BLjyI

pathlib.Path, parent, name and suffix, and joining paths with the / operator.



What does this package do? - Episode 2: Enhanced Dictionaries (defaultdict and Counter)

PyCharm, a JetBrains IDE (Integrated Development Environment) · YouTube · 9:53 · youtube.com/watch?v=I3ZnopcpYY0

Grouping with defaultdict and counting with Counter, as in the last part of loader.py.

Real-life example 5.2 — four damaged lines

THE TASK

Load the twelve rows of September 14 that the office manager exported (`routines_sample_broken.csv`), four of them damaged — a decimal `17.0`, a trailing comma, a date typed `2026/09/14` and a sign typo `-19` — report each bad line with its number and reason without stopping; then load the full 420-row dataset with the same loader, print the first three entries, group the minutes per routine and count the runs per tool.

Demo: `code/loader.py` · interactive replay: `code/demo52_loader.html`

DELIVERABLES OF THE EXAMPLE

- `jarvis/loader.py` with `DataLoader.load(path) -> list[TimeEntry]` and the `skipped` report
- The console report: `sample: loaded 8, skipped 4` and the four reasons with their line numbers
- `full file: loaded 420, skipped 0` and the first three entries — the output of `python -m jarvis load`
- The grouped minutes (6 routines × 70 runs) and the runs per tool (370 / 35 / 15)

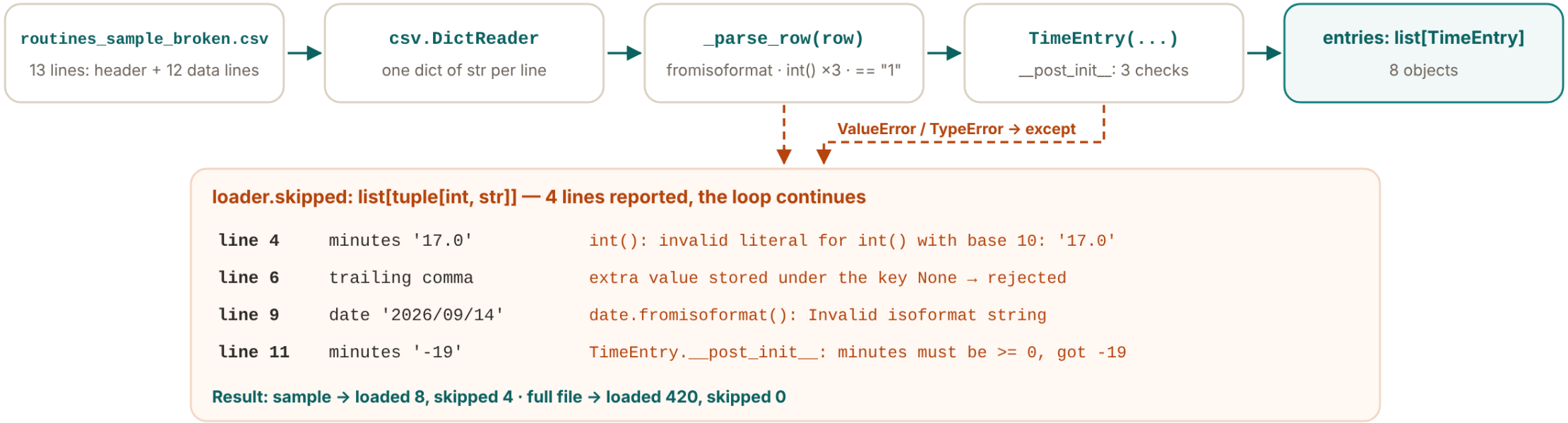
WHERE IT RETURNS (FUTURE BRICKS)

- Brick 5 (due Oct 29) — `loader.py` and `python -m jarvis load`
- Brick 6 (due Nov 5) — grouping per routine, staff member and day; the Store imports the entries
- Brick 10 (due Dec 10) — three bad-input cases demonstrated, never a crash
- Mid-Term Part B (Nov 20) — read and complete code like `loader.py`

Figure 5.2 — The loader pipeline

Figure 5.2a — The loader pipeline

routines_sample_broken.csv through DataLoader.load()



Each data line becomes a dict of strings, each field is converted to its type, and TimeEntry checks the values. A conversion or check that fails sends the line to the skipped report with its line number and reason. Right: what the full file gives after loading — 70 runs per routine and the runs per tool. *Files: figures/fig52a_pipeline.svg and figures/fig52a_pipeline.png*

5.3 Responsibilities → classes → UML (Unified Modeling Language) — key terms

Responsibility

Something a class must know (information it keeps) or do (an operation it performs) for the system.

Single Responsibility Principle (SRP)

A class should have one responsibility; in Robert C. Martin's wording, a module should have one, and only one, reason to change.

Class diagram

A **UML** diagram that shows classes as boxes with three compartments (name, attributes, operations) and the relationships between them.

Aggregation

A whole–part association in which the part can exist without the whole; drawn with a hollow diamond at the whole.

Composition

A whole–part association in which the part belongs to one whole at a time and is deleted with it; drawn with a filled diamond at the whole.

Multiplicity

The number of objects at one end of a relationship: `1`, `0..1`, `*` (same as `0..*`), `1..*`.

From requirements to responsibilities

- **Responsibility**: what a class must **know** or **do**
- Each requirement → exactly **one** class
- **SRP** (Single Responsibility Principle): one responsibility = one reason to change (Robert C. Martin)
- **Cohesion** high: every member serves the same job
- **Coupling** low: a change rarely spreads
- **CRC** (Class–Responsibility–Collaborator) cards (Beck & Cunningham, 1989): class · responsibilities · collaborators

KEY IDEA

Give each requirement to exactly one class; one responsibility per class means one reason to change.

The JARVIS classes and their modules (1/2)

Table 5.3.A (1/2) · file tables/t53_responsibilities.csv

Class	Module	Responsibility (one sentence)	Built in
`TimeEntry`	`models.py`	holds one timed routine run and computes its cost	Class 5 · Brick 5
`Task`	`models.py`	holds one to-do item: title, due date, priority, done	Class 5 · Brick 5
`DataLoader`	`loader.py`	turns a routines **CSV** (Comma-Separated Values) file into `TimeEntry` objects and reports bad rows	Class 5 · Brick 5
`Store`	`store.py`	owns the database file `data/jarvis.db`: save, query, export	Class 6 · Brick 6
`StatsEngine`	`stats_engine.py`	computes the statistics of a list of minutes	Class 6 · Brick 6



The JARVIS classes and their modules (2/2)

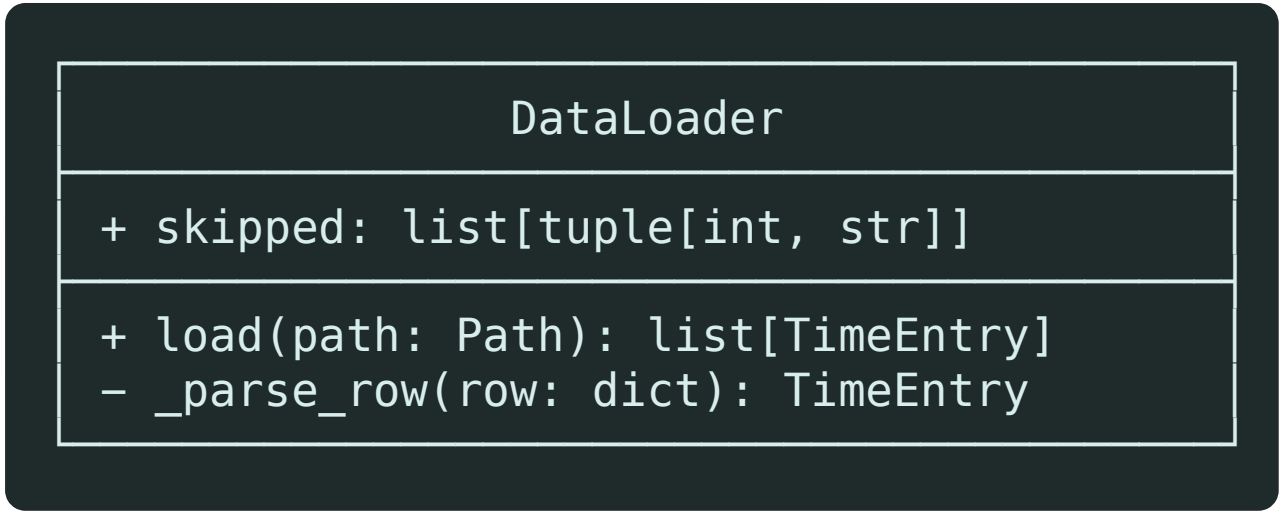
Table 5.3.A (2/2) · file tables/t53_responsibilities.csv

Class	Module	Responsibility (one sentence)	Built in
`EconEngine`	`econ_engine.py`	prices the time of the routines: cost, break-even, payback	Class 6 · Brick 6
`Tool` (abstract)	`tools/base.py`	defines one action the assistant can perform	skeleton Brick 5 · Brick 7
`Provider` (abstract)	`providers.py`	turns a text request into an `Intent`	skeleton Brick 5 · Brick 7
`Agent`	`agent.py`	runs the loop request → intent → tool → result → log	Class 7 · Brick 7
`Reporter`	`reporter.py`	writes the console report, the charts and the dashboard	Class 6 · Brick 11



Reading a UML (Unified Modeling Language) class box

- **UML**: **OMG** (Object Management Group) standard, version 2.5.1
- 1 · class name (*italic* = abstract)
- 2 · attributes `visibility name: type`
- 3 · operations `visibility name(param: type): return`
- `+` public · `–` private · `#` protected · `~` package
- `{readOnly}` = property without setter



Relationships between classes



Association — solid line; the arrowhead shows the direction of navigation

Agent → Store ("logs runs in"): the agent keeps a reference to the store.



Aggregation — hollow diamond at the whole; the part can exist without it

Agent ◇— Tool: `main()` creates the tools and passes them in.



Composition — filled diamond at the whole; the part belongs to one whole and is deleted with it

Only if `Agent.__init__` created its own provider — JARVIS avoids it on purpose.



Dependency — dashed arrow; one class uses another for a moment

DataLoader ⇢ TimeEntry ("creates"): builds entries and returns them.



Generalization — solid line, hollow triangle at the superclass

LogTimeTool —▷ Tool: a LogTimeTool **is a** Tool (inheritance).

Multiplicity

- `1` exactly one · `0..1` zero or one
- `*` = `0..*` any number, zero included · `1..*` at least one
- `Agent "1" ◇— "1..*" Tool` → one agent holds **one or more** tools
- `Store "1" — "0..*" TimeEntry` → one store keeps **zero or more** entries
- An empty database is a **valid** state

KEY IDEA

1, 0..1, * (0..*), 1..*: how many objects take part at one end of a relationship, read from the other side.

Drawing the diagram: draw.io, PlantUML or paper

- **draw.io** (diagrams.net): boxes by hand, in the browser
- **PlantUML**: text → diagram; `docs/uml.puml` in Git
- A clean photographed drawing is accepted

```
@startuml
skinparam classAttributeIconSize 0
abstract class Tool {
  + name: str
  + run(args: dict, store: Store): str
    {abstract}
}
Tool <|-- LogTimeTool
Agent "1" o-- "1..*" Tool : dispatches to
Agent "1" --> "1" Store : logs runs in
DataLoader ..> TimeEntry : creates
@enduml
```




Activity 3 — Zen break + six UML (Unified Modeling Language) cards (0:41–0:45)

⏱️ 0:41–0:45 · Zen break + flash-cards

1. 2 minutes of zen: stand up, stretch, breathe in for 4 seconds, hold 4, breathe out 4.
2. In pairs, flash-cards (6): `−`, `+`, association, aggregation, inherits, multiplicity `1..*`.
3. One student shows the card, the other gives the meaning and one JARVIS example; then swap.

Videos for home — Lesson 5.3

For home, not for class. Each video is public and shorter than 15 minutes (checked October 17, 2026); videos are third-party content — sections 14 and 15 of the Cyber.SoHo Legal Notice apply.

UML (Unified Modeling Language) class diagrams

Lucid Software · YouTube · 12:24 · youtube.com/watch?v=6XrL5jXmTwM

Attributes, methods, visibility, inheritance and abstract classes, association, aggregation and composition, multiplicity.

Class diagram arrows: Association, Aggregation, Composition, Dependency, Multiplicity, Inheritance

Gleek diagram maker · YouTube · 4:01 · youtube.com/watch?v=KL9au7z3MpU

A four-minute recap of every relationship symbol of Lesson 5.3.

CRC (Class–Responsibility–Collaborator) Card Analysis Example

Paul Gestwicki · YouTube · 11:07 · youtube.com/watch?v=otKUer13HnA

A university lecture that derives classes, responsibilities and collaborators from a user story.

#1: SOLID (Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion) Design Principle with Python : Single Responsibility Principle | SRP

sekhar srinivas · YouTube · 9:33 · youtube.com/watch?v=acmPg6aV-Zs

The **SRP** applied to Python classes. **SOLID** is the name of a group of five object-oriented design principles; the **SRP** is the first of them.

Real-life example 5.3 — the owner's dashboard request

THE TASK

Give each of the twelve JARVIS requirements (taken from the user stories of Brick 2) to exactly one class of the 16-class reference design, detect a class that received two responsibilities, correct the assignment, and generate the **UML** (Unified Modeling Language) class diagram as PlantUML text (`jarvis_uml.puml`).

Demo: `code/design_to_uml.py` · interactive replay: `code/demo53_design_to_uml.html`

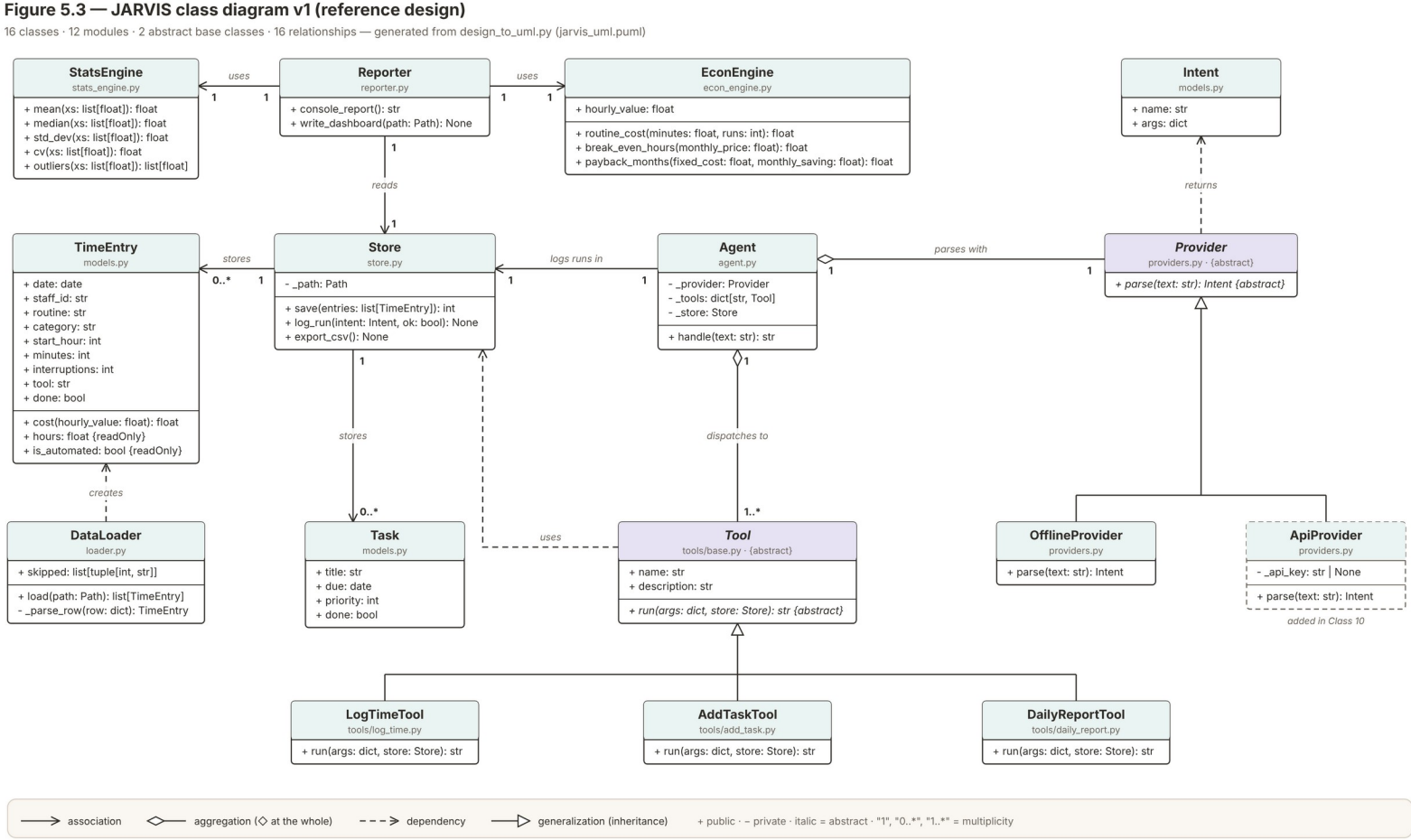
DELIVERABLES OF THE EXAMPLE

- The responsibility table: requirement → class → module
- The check output: the **SRP** (Single Responsibility Principle) warning on the draft, then "OK" after the correction
- `jarvis_uml.puml` (PlantUML text, 102 lines) and Figure 5.3 (the rendered diagram)
- Two or three sentences for `docs/design.md`: why the dashboard belongs to the Reporter

WHERE IT RETURNS (FUTURE BRICKS)

- Brick 5 (due Oct 29) — **UML** class diagram v1 and `docs/design.md`
- Alpha demo (Nov 6) — the design slide; its **PDF** (Portable Document Format) file is submitted with Brick 7 (due Nov 12)
- Class 8 (Nov 13) — the Design Card, allowed at the Mid-Term of Nov 20
- Mid-Term Part B (Nov 20) — read a **UML** fragment, explain one association
- Brick 12 (due Sunday, Dec 20) — report §4 Design

Figure 5.3 — JARVIS class diagram v1 (reference)



WHAT THE FIGURE SHOWS

Sixteen classes in twelve modules. Tool and Provider are abstract (italic names); five subclasses inherit from them. The Agent aggregates one Provider and one or more Tools and logs every run in the Store; the Store keeps zero or more entries and tasks; the Reporter reads the Store and uses both engines. Your Brick 5 diagram adds the method signatures of your own Bricks 3 and 4.

Files: figures/fig53_jarvis_uml.svg and figures/fig53_jarvis_uml.png

5.4 ABCs (Abstract Base Classes), polymorphism & clean code — key terms

Inheritance

A subclass receives the attributes and methods of its superclass and can add new ones or replace them: `class LogTimeTool(Tool):`.

Overriding

A subclass defines a method with the same name and parameters as a method of its superclass; for objects of the subclass, the subclass version runs.

Abstract base class (ABC)

A class that declares methods its subclasses must implement and that cannot be instantiated itself; in Python it inherits from `abc.ABC`.

Polymorphism

The same call — `tool.run(args, store)`, `provider.parse(text)` — executes different code depending on the class of the object that receives it.

Dependency injection

Passing the objects a class needs (provider, tools, store) as parameters of its constructor instead of creating them inside the class.

Code smell

A surface sign in the code that usually points to a deeper problem (Martin Fowler): a secret in the code, a hard-coded path, a 200-line function.

Inheritance and overriding

- `class LogTimeTool(Tool):` — **superclass** `Tool`, **subclass** `LogTimeTool`
- **Overriding**: the subclass writes its own `run()`

- "A LogTimeTool **is a** Tool" → inheritance
- "An Agent is a Tool" is false → the Agent **has** tools

```python
class LogTimeTool(Tool):
    """Records the minutes of one routine run."""

    name = "log_time"
    description = "log the minutes spent on one routine"

    def run(self, args: dict, store: MemoryStore) -> str:
        store.entries.append({"routine": args["routine"], "minutes": args["minutes"]})
        return f"logged {args['minutes']} min of {args['routine']}"
```




Abstract base classes: Tool and Provider

- `class Tool(ABC)` + `@abstractmethod run(...)`
- `Tool()` → **TypeError**: Can't instantiate abstract class
- `SummarizeTool` without `run()` → TypeError **at creation**
- **PEP** (Python Enhancement Proposal) 3119

```python
class Tool(ABC):
    name: str = ""
    description: str = ""

    @abstractmethod
    def run(self, args: dict, store: MemoryStore) -> str:
        """Perform the action and return a reply."""


class Provider(ABC):
    @abstractmethod
    def parse(self, text: str) -> Intent:
        """Return the Intent of the request."""
```




Polymorphism in the agent loop

- One call, different code: `tool.run(intent.args, self._store)`
- No `if intent.name == ...: elif ...` chain
- New tool (Class 10) = one subclass + one list item + one provider rule
- ApiProvider (Class 10) = one argument changed in `main()`

```
def handle(self, text: str) -> str:
    intent = self._provider.parse(text)      # any Provider subclass
    tool = self._tools.get(intent.name)
    if tool is None:
        return f"sorry, no tool for {text!r}"
    reply = tool.run(intent.args, self._store)  # any Tool subclass
    self._store.runs.append((text, intent.name, type(tool).__name__))
    return reply
```




Dependency injection — and what the rubric says

- `Agent(OfflineProvider(), [LogTimeTool(), AddTaskTool(), DailyReportTool()], store)`
- **Dependency injection**: objects passed in, not created inside
- A test passes a fake provider or a memory store
- OfflineProvider first: tests, Mid-Term, defense never need a network or an **API** (Application Programming Interface) key
- Rubric, Tech Implementation — **Exemplary**: “advanced **OOP** (Object-Oriented Programming) (abstract base classes, polymorphism) used effectively”
- **Developing**: “procedural style instead of **OOP**”

KEY IDEA

The Agent receives its provider, tools and store; tests pass fakes; the offline provider keeps every demo independent of the network.

Clean-code habits that cost or earn marks (1/2)

Table 5.4.A (1/2) · file tables/t54_clean_code.csv

Habit	The smell it prevents	Rule in JARVIS
Follow **PEP** (Python Enhancement Proposal) 8	inconsistent indentation, 150-character lines, names in mixed styles	4 spaces; at most 79 characters per line of code and 72 for comments and docstrings; `snake_case` for functions and variables, `CapWords` for classes, `UPPER_CASE` for constants
Type hints on public functions (**PEP** 484)	a reader cannot tell what a function accepts or returns	`def load(self, path: Path) -> list[TimeEntry]:`
A docstring on every module, class and public method (**PEP** 257)	code whose purpose must be guessed	one sentence that states the responsibility
Small functions, one job each	an 80-line function that loads, computes and prints	if the docstring needs “and”, split the function
Meaningful names	`x1`, `x2`, `l`, `tmp`	`minutes_manual`, `minutes_jarvis`, `line_no`



Clean-code habits that cost or earn marks (2/2)

Table 5.4.A (2/2) · file tables/t54_clean_code.csv

Habit	The smell it prevents	Rule in JARVIS
No secret in the code	`API_KEY = "demo-key-…"` committed to GitHub	`os.environ.get("JARVIS_API_KEY")`, a git-ignored `.env`, a repository secret; a key committed once is replaced the same day
No hard-coded path	`C:\Users\ana\Desktop\log.csv`	`Path(__file__).resolve().parent` (Lesson 5.2)
No mutable default argument	`def add(item, seen=[])`: one list shared by every call	`seen=None`, then `seen = [] if seen is None else seen`
Catch only the exceptions you expect	`except:` or `except Exception:` hides real bugs	`except (ValueError, TypeError)` around the conversions
A design that pytest can test	hidden global state, objects created deep inside methods	inputs as parameters, results returned, dependencies injected



Activity 4 — Spot the four smells (0:56–1:00)

⏱️ 0:56–1:00 · Fun quiz

1. Read the ten lines on screen (2 minutes, in pairs).
2. Name four smells and the line of each one.
3. Bonus: find the line that crashes the program on the first line of the file.

```
API_KEY = "demo-key-7Q2xK9-not-real"

def load(p="C:\\Users\\ana\\Desktop\\log.csv", rows=[]):
    f = open(p)
    for l in f:
        x1 = l.split(",")
        x2 = int(x1[5])
        rows.append(x2)
    return rows
print(load(), load())
```

YOUR GOAL

Four smells in ten lines: find them, name them, fix them — and find the line that crashes.

Videos for home — Lesson 5.4

For home, not for class. Each video is public and shorter than 15 minutes (checked October 17, 2026); videos are third-party content — sections 14 and 15 of the Cyber.SoHo Legal Notice apply.



Learn Python ABSTRACT CLASSES in 7 minutes!

Bro Code · YouTube · 7:02 · youtube.com/watch?v=97V7ICVeTJc

ABC (Abstract Base Class), @abstractmethod, and why an abstract class cannot be instantiated.



What is Polymorphism in Python?

Indently · YouTube · 7:38 · youtube.com/watch?v=AfBc7yyigl4

Polymorphism with an abstract base class and several subclasses used through one function — the pattern of Agent.handle().



25 nooby Python habits you need to ditch

mCoding · YouTube · 9:12 · youtube.com/watch?v=qUeud6DvOWI

Code habits that cost marks, including the mutable default argument with its None fix, bare except and **PEP** (Python Enhancement Proposal) 8.



How to Use Environment Variables to Hide Passwords and Secret Data

KenBroTech · YouTube · 7:55 · youtube.com/watch?v=NDbr32xMUDQ

Keeping a secret such as an **API** (Application Programming Interface) key out of the code, in an environment variable.

Real-life example 5.4 — S2's morning requests

THE TASK

Build an `Agent` that receives five text requests from a staff member — “log 25 min email_triage”, “add task call the accountant”, “log 12 min standup_notes”, “report” and “book a flight” — and sends each one through an `OfflineProvider` and the matching `Tool` subclass, using only the abstract types `Provider` and `Tool`; record which class handled each request, and show that an unfinished tool (`SummarizeTool`) cannot even be created.

Demo: `code/agent_dispatch.py` · interactive replay: `code/demo54_agent_dispatch.html`

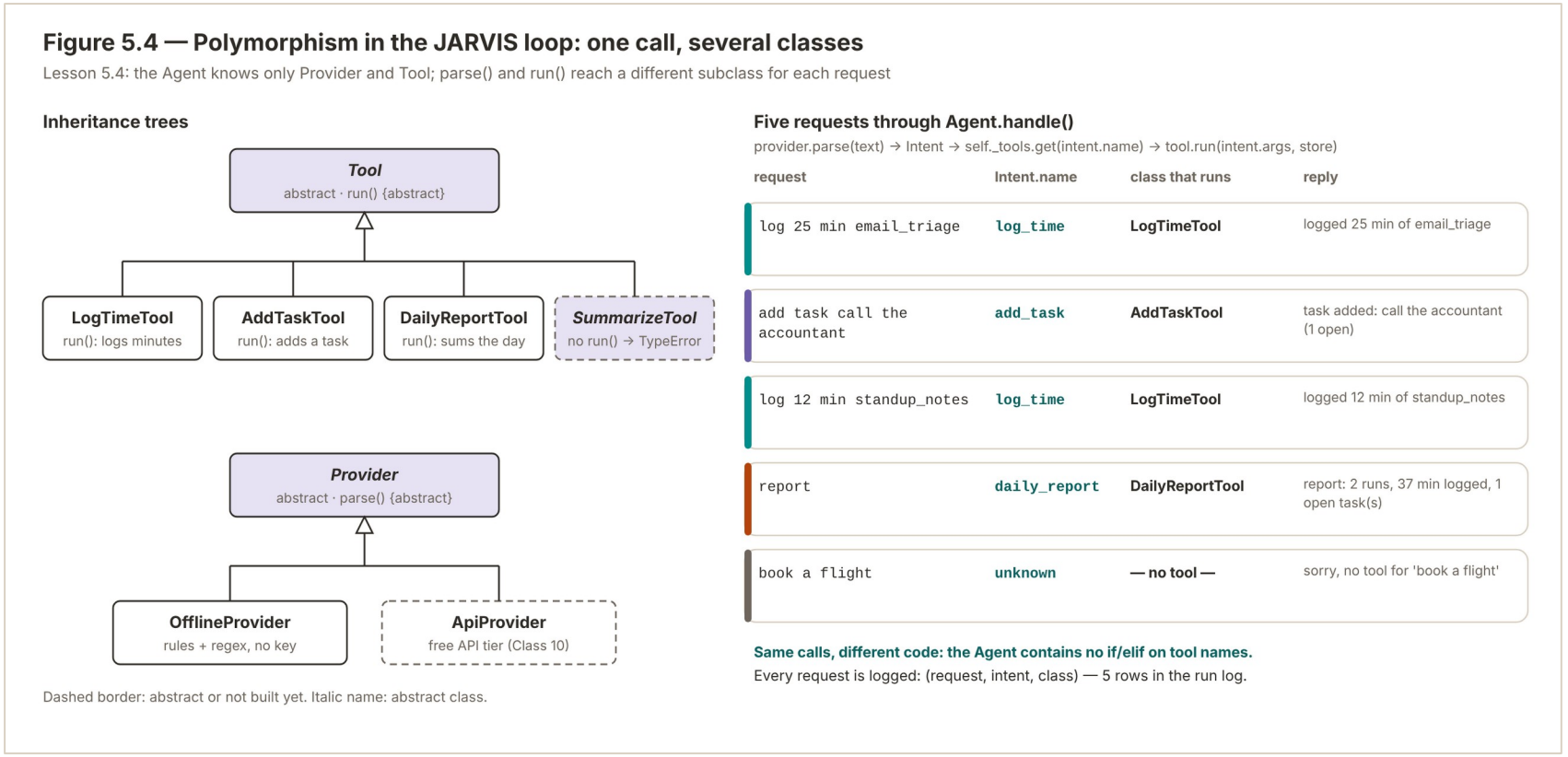
DELIVERABLES OF THE EXAMPLE

- `tools/base.py` (`Tool`, abstract) and `providers.py` (`Provider`, abstract) — the Brick 5 skeletons
- `OfflineProvider` with its regular expression, `LogTimeTool`, `AddTaskTool`, `DailyReportTool` and the `Agent`
- The five replies and the run log (request, intent, class that handled it)
- The `TypeError` of `SummarizeTool()` — the first test of `tests/test_tools.py`

WHERE IT RETURNS (FUTURE BRICKS)

- Brick 5 (due Oct 29) — the two abstract base classes as skeletons
- Brick 7 (due Nov 12) — OfflineProvider, Tool base class, two tools wired into the Agent
- Brick 9 (due Dec 3) — tests for the tools and the provider
- Brick 10 (due Dec 10) — ApiProvider as one more subclass, with the offline fallback; code-review swap
- Final defense (Jan 22) — “polymorphic Tool and Provider” is evidence for the Exemplary level

Figure 5.4 — Polymorphism in the JARVIS loop



Left: the two inheritance trees; Tool and Provider are abstract, SummarizeTool is still abstract because it has no run(). Right: the five requests of the real-life example — the same calls parse() and run() reach a different subclass each time, and every request is recorded in the run log. *Files: figures/fig54_polymorphism.svg and figures/fig54_polymorphism.png*

Workshop — Brick 5 “Design Pack + Working Loader”

Time	What happens in the workshop
1:00–1:10	Brief; agree the class list of your team (8–10 classes across 6–7 modules).
1:10–1:40	**UML** (Unified Modeling Language) class diagram v1 (draw.io, PlantUML or a clean hand drawing, photographed) with the attributes and the method signatures of your Bricks 3–4.
1:40–2:30	Code: `jarvis/models.py` (`TimeEntry`, `Task`); `jarvis/loader.py` with `load(path: Path) -> list[TimeEntry]`; `python -m jarvis load` prints the row count, the skipped count and the first three entries; a bad row is reported and skipped, never a crash.
2:30–2:50	Skeletons: `stats_engine.py` and `econ_engine.py` with signatures and docstrings whose bodies raise `NotImplementedError("TODO")`; `store.py` with the SQLite schema (entries, tasks, runs) in a `SCHEMA` string and an empty `Store` class; `tools/base.py` and `providers.py` with the two abstract base classes.
2:50–3:00	Checkpoint: run on the full dataset; commit; tag `alpha-0.1`.

Brick 5 — Design Pack + Working Loader

Due: Thursday, October 29, 2026, 23:59 — **LMS** (Learning Management System) + GitHub (tag alpha-0.1)

- **UML** class diagram v1 (`docs/uml.png` or `docs/uml.puml`)
- `models.py`, `loader.py`, `__main__.py` running on the dataset
- Engine, Store, Tool and Provider skeletons with docstrings; the SQLite schema
- `docs/design.md` (≤ 1 page) and the updated `README.md`
- Weekly Progress Report #5

Feeds into: Technical report §4 Design · Mid-Term Part B (**UML**, encapsulation, reading code) · the Design Card

Mid-Term (Nov 20): format and allowed materials

- **Date:** Friday, November 20, 2026 — 30 % of the final grade, individual, in class, 120 minutes.
- **Part A — Computational (≈ 60 min):** routine cost, marginal benefit and marginal cost, break-even hours, payback, mean, median, mode, sample **SD** (standard deviation), correlation and a before/after comparison, on a new slice of the routines data.
- **Part B — Application (≈ 60 min):** complete or fix a Python method, explain a **UML** (Unified Modeling Language) fragment (why `Tool` is an abstract base class, one association), write two user stories and one research question.
- **Allowed materials:** one handwritten Formula Card and one handwritten Design Card (one page each), a pen and a calculator. No laptop, no phone.
- Your Design Card is today's work: the class diagram and one sentence of responsibility per class.



ALLOWED AT THE MID-TERM

One handwritten **Formula Card** and one handwritten **Design Card**, a pen and a calculator. Nothing else: no laptop, no phone, no printed notes.

Home work and next class

1. Finish Brick 5; complete the `README.md` section “How to run”: `python -m venv .venv`, `pip install -r requirements.txt`, `python -m jarvis load` — relative paths only.
2. Write `docs/design.md` (one page at most): why these classes, what each one owns, which classes are abstract and why.
3. Skim Leedy & Ormrod, Chapter 5, *Writing the Research Proposal* (20 minutes).
4. Weekly Progress Report #5. From this week, also log the hours you spend on JARVIS setup: they are the setup cost of the economic model of Brick 3.
5. **Next:** Development I — The Engines and the Store Come Alive



NEXT CLASS

Class 6 — Friday, October 30, 2026. `StatsEngine` and `EconEngine` implemented and validated against your Phase 1 scripts; grouping per routine, staff member and day; the SQLite `Store` with the tiny-database rule; console reporting. Brick 6 is due Thursday, November 5 (tag alpha-0.2). Bring today's `models.py` and `loader.py`: Class 6 builds on them.

All links of this module (1/3)

Each domain was checked with Google Safe Browsing on October 17, 2026; re-check a link on VirusTotal (virustotal.com) before you open it — Annex C of the Legal Notice. What each link is for: hand-out, “All links”.

- **Python Software Foundation (PSF)**
  python.org/psf
- **Python documentation (home)**
  docs.python.org/3
- **dataclasses — Data Classes** (Lesson 5.1)
  docs.python.org/3/library/dataclasses.html
- **The Python Tutorial — Classes, Private Variables** (Lesson 5.1)
  docs.python.org/3/tutorial/classes.html#private-va…
- **Built-in functions — property and int** (Lesson 5.1)
  docs.python.org/3/library/functions.html#property
- **csv — CSV** (Comma-Separated Values) **File Reading and Writing** (Lesson 5.2)
  docs.python.org/3/library/csv.html#csv.DictReader
- **datetime — date.fromisoformat** (Lesson 5.2)
  docs.python.org/3/library/datetime.html#datetime.d…
- **pathlib — Object-oriented filesystem paths** (Lesson 5.2)
  docs.python.org/3/library/pathlib.html
- **collections — defaultdict and Counter** (Lesson 5.2)
  docs.python.org/3/library/collections.html#collect…
- **Glossary — EAFP** (Easier to Ask for Forgiveness than Permission) (Lesson 5.2)
  docs.python.org/3/glossary.html#term-EAFP
- **abc — Abstract Base Classes** (Lesson 5.4)
  docs.python.org/3/library/abc.html
- **The Python Tutorial — Default Argument Values** (Lesson 5.4)
  docs.python.org/3/tutorial/controlflow.html#defaul…
- **os.environ** (Lesson 5.4)
  docs.python.org/3/library/os.html#os.environ

All links of this module (2/3)

Each domain was checked with Google Safe Browsing on October 17, 2026; re-check a link on VirusTotal (virustotal.com) before you open it — Annex C of the Legal Notice. What each link is for: hand-out, “All links”.

- **re — Regular expression operations** (Lesson 5.4)
  docs.python.org/3/library/re.html
- **PEP** (Python Enhancement Proposal) **8 — Style Guide for Python Code** (Lesson 5.4)
  peps.python.org/pep-0008
- **PEP 257 — Docstring Conventions**
  peps.python.org/pep-0257
- **PEP 484 — Type Hints**
  peps.python.org/pep-0484
- **PEP 557 — Data Classes**
  peps.python.org/pep-0557
- **PEP 3119 — Introducing Abstract Base Classes**
  peps.python.org/pep-3119
- **PEP 428 — The pathlib module**
  peps.python.org/pep-0428
- **Object Management Group (OMG)**
  omg.org
- **UML** (Unified Modeling Language) **specification, version 2.5.1 (OMG)** (Lesson 5.3)
  omg.org/spec/UML
- **PlantUML — Class diagram syntax**
  plantuml.com/class-diagram
- **PlantUML online editor**
  editor.plantuml.com
- **draw.io (diagrams.net)**
  drawio.com
- **Beck & Cunningham (1989), A Laboratory for Teaching Object-Oriented Thinking** (Lesson 5.3)
  c2.com/doc/oopsla89/paper.html

All links (3/3), legal and thanks

Each domain was checked with Google Safe Browsing on October 17, 2026; re-check a link on VirusTotal (virustotal.com) before you open it — Annex C of the Legal Notice. What each link is for: hand-out, “All links”.

- **Robert C. Martin (2014), The Single Responsibility Principle** (Lesson 5.3)
  blog.cleancoder.com/uncle-bob/2014/05/08/SingleRep…
- **Martin Fowler (2006), Code Smell** (Lesson 5.4)
  martinfowler.com/bliki/CodeSmell.html
- **ISO** (International Organization for Standardization) **8601 — Date and time format (ISO)** (Lesson 5.2)
  iso.org/iso-8601-date-and-time-format.html
- **International Organization for Standardization (ISO)**
  iso.org
- **Getting Started with Python in VS Code** (Visual Studio Code)
  code.visualstudio.com/docs/python/python-tutorial
- **GitHub Docs**
  docs.github.com
- **Leedy & Ormrod (2019), Practical Research, 12th ed. (Pearson)**
  pearson.com/en-us/subject-catalog/p/practical-rese…
- **VirusTotal — URL** (Uniform Resource Locator) **scan**
  virustotal.com/gui/home/url
- **Google Safe Browsing — site status**
  transparencyreport.google.com/safe-browsing/search
- **Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (v2.0)**
  cyber-soho.github.io/prevmon/00.01.%20cyber.soho%2…

LEGAL — CYBER.SOHO LEGAL NOTICE, TERMS OF USE AND ACCEPTABLE USE POLICY

► Legal Notice & Terms of Use ► § 17 Recording and sharing
► § 18 Academic integrity ► Annex C Link safety checklist

Thank you.

Workshop next: Brick 5, due Thursday, October 29, 2026, 23:59.